

Off-Policy Temporal-Difference Control: Q-Learning

Reinforcement Learning

Sai Sampath Kedari

sampath@umich.edu

Contents

1	Setup and What Is New	2
2	From Operator to Sample	2
2.1	The Two Forms of q^*	2
2.2	Rewriting the Bellman Optimality Equation as a Conditional Expectation	2
2.3	Replacing the Expectation with One Sample	4
2.4	Running-Mean Update	6
3	Off-Policy: What the Change of Operator Bought Us	6
4	The Algorithm	7
5	Convergence	7

1 Setup and What Is New

We continue with the infinite-horizon discounted MDP of the previous reports: finite state space $\mathcal{S}$, finite action sets $\mathcal{A}(s)$, joint transition kernel $p(s', r \mid s, a)$, discount $\gamma \in [0, 1)$, and stationary policies $\pi(a \mid s)$.

The SARSA(0) report converted the Bellman *expectation* operator T_π on $\mathcal{Q}$ into a sampled update. Its fixed point is q_π , the value of whatever policy we are currently following, so the conversion produces an algorithm that estimates the value of the agent's own policy. To make this useful for control, the policy is improved ε -greedily after every update; SARSA evaluates and improves *the same* policy.

This report changes one thing: the operator. We start from the Bellman *optimality* operator T on $\mathcal{Q}$, whose fixed point is q^* directly. There is no policy whose value we are estimating; the operator targets the optimal action-value function directly, the same way value iteration does in the model-based setting. Applying the same statistical move (replace each expectation with one sample) to this operator gives Q-learning. The single algebraic change $\sum_{a'} \pi(a' \mid s') \rightarrow \max_{a'}$ in the operator has three consequences that fall out automatically: only one expectation survives, the update uses a quadruple instead of a quintuple, and the algorithm is off-policy.

2 From Operator to Sample

2.1 The Two Forms of q^*

The optimal action-value function is the fixed point of the Bellman optimality operator on $\mathcal{Q}$:

$$q^*(s, a) = (Tq^*)(s, a) = r(s, a) + \gamma \sum_{s'} p(s' \mid s, a) \max_{a'} q^*(s', a').$$

The corresponding fixed-point iteration in DP is value iteration on Q :

$$Q_{k+1}(s, a) = (TQ_k)(s, a) = r(s, a) + \gamma \sum_{s'} p(s' \mid s, a) \max_{a'} Q_k(s', a'),$$

with $Q_k \rightarrow q^*$ for any Q_0 . The operator equation looks structurally similar to the SARSA(0) operator equation, but with $\sum_{a'} \pi(a' \mid s')$ replaced by $\max_{a'}$. That single change is the whole story of the report.

2.2 Rewriting the Bellman Optimality Equation as a Conditional Expectation

We have the Bellman optimality equation for q^* ,

$$q^*(s, a) = r(s, a) + \gamma \sum_{s'} p(s' \mid s, a) \max_{a'} q^*(s', a').$$

We rewrite it as a single conditional expectation, so that one trajectory step gives one realization of the integrand. There are only two transformations: the outer state sum, and folding in the reward. The inner $\max_{a'}$ is *not* a sum-with-weights, so it does not become an expectation. That single fact is the structural pivot of the entire report, and we address it explicitly after Transformation 1.

Transformation 1: the outer state sum. The outer sum is over next states s' , weighted by the environment transition $p(s' | s, a)$. Naming the random draw S_{t+1} ,

$$\sum_{s'} p(s' | s, a) \max_{a'} q^*(s', a') = \mathbb{E}_{S_{t+1} \sim p(\cdot | s, a)} \left[\max_{a'} q^*(S_{t+1}, a') \right].$$

The variable s' has been replaced by S_{t+1} , so the bracketed quantity is random through S_{t+1} . But the max itself, at any fixed value of S_{t+1} , is not.

The inner max is deterministic. In SARSA(0), the inner action term $\sum_{a'} \pi(a' | s') q_\pi(s', a')$ was a probability-weighted average, so it converted to an expectation $\mathbb{E}_{A_{t+1} \sim \pi}[q_\pi(s', A_{t+1})]$ and introduced a second random variable.

In Q-learning, the inner term is $\max_{a'} q^*(s', a')$. No probability distribution over a' . The max is a deterministic function of s' and the table q^* : given those inputs, it returns the largest entry of the row $q^*(s', \cdot)$. Nothing to average over, nothing to sample. A table lookup followed by a max. Three consequences follow, and they shape the rest of the report:

- Only one expectation survives in the full form (environment only, not environment-and-policy).
- The update uses a quadruple $(S_t, A_t, R_{t+1}, S_{t+1})$, not a quintuple, because there is no second sample.
- The algorithm is off-policy: the eventual target references no policy, since the operator references no policy.

This is what the slogan “Q-learning is SARSA with $A_{t+1} \sim \pi$ replaced by $\arg \max_{a'} Q(S_{t+1}, a')$ ” really says. The replacement comes from the operator: a max instead of a sum-with-weights, so no expectation, so no sample.

Transformation 2: folding in the reward. With Transformation 1 applied,

$$q^*(s, a) = r(s, a) + \gamma \mathbb{E}_{S_{t+1} \sim p(\cdot | s, a)} \left[\max_{a'} q^*(S_{t+1}, a') \right].$$

The expected reward $r(s, a) = \mathbb{E}[R_{t+1} | s, a]$ is under the joint $p(\cdot, \cdot | s, a)$, whose next-state marginal is the same $p(\cdot | s, a)$ governing S_{t+1} . Folding the two expectations together gives the **full notational form**:

$$q^*(s, a) = \mathbb{E}_{(S_{t+1}, R_{t+1}) \sim p(\cdot, \cdot | s, a)} \left[R_{t+1} + \gamma \max_{a'} q^*(S_{t+1}, a') \right].$$

What we have. Random variables: R_{t+1} and S_{t+1} , jointly from the environment. No A_{t+1} .

Sampling: one draw. No second step.

Policy: absent. No π subscript on $\mathbb{E}$.

Compact form.

$$q^*(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} q^*(S_{t+1}, a') \mid S_t = s, A_t = a \right].$$

Compare with SARSA(0)'s $q_\pi(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid s, a]$. The subscript changes (π vs none), the bootstrap changes ($q_\pi(\cdot, A_{t+1})$ vs $\max_{a'} q^*$), and the random variables change ((S, R, A) vs (S, R)). All three are one fact in different clothes.

2.3 Replacing the Expectation with One Sample

The conditional expectation has one expectation to sample and one bootstrap to evaluate.

Draw the expectation. Execute action a in state s ; the environment returns one realization $(S_{t+1}, R_{t+1}) \sim p(\cdot, \cdot \mid s, a)$.

Evaluate the integrand. The first term R_{t+1} is observed. The second is $\max_{a'} q^*(S_{t+1}, a')$; since q^* is unknown, we bootstrap with the current estimate Q , giving $\max_{a'} Q(S_{t+1}, a')$. This is a table lookup followed by a max — no sampling, no policy.

The Q-learning target.

$$\widehat{q^*}(s, a) = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a').$$

The update is built from the quadruple $(S_t, A_t, R_{t+1}, S_{t+1})$. Two fixed, two sampled, no A_{t+1} .

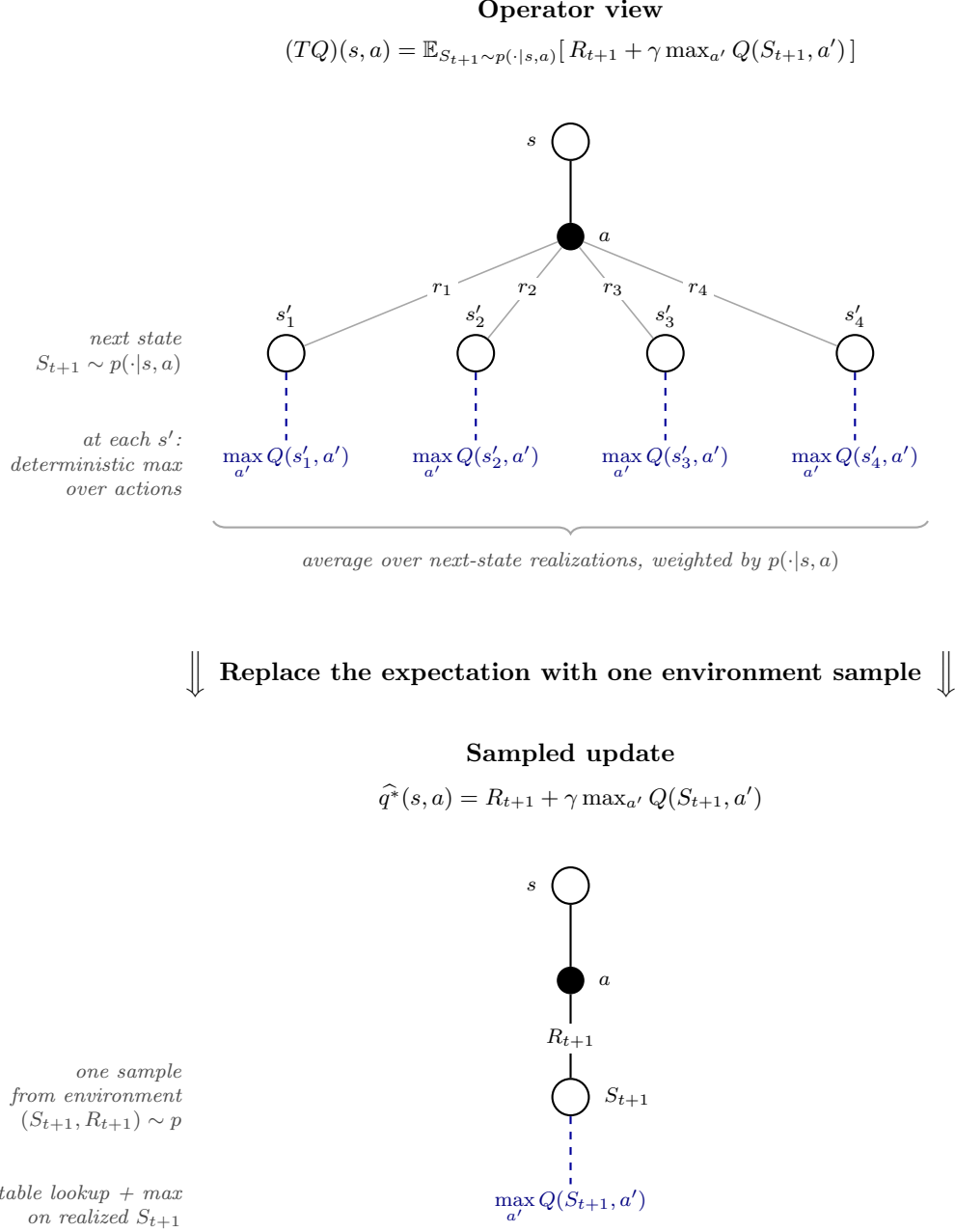


Figure 1: Operator-to-sample conversion in Q-learning. Empty circles are states, filled circles are actions; solid edges are real transitions, blue dashed edges show the deterministic max evaluation. *Top*: the optimality operator at (s, a) averages over every possible next state s' (weighted by p), and at each s' reads off $\max_{a'} Q(s', a')$ directly from the current Q -table. The action side is not a branching: the max is a deterministic readout, not an expectation. *Bottom*: Q-learning replaces the environment average with one sampled transition (S_{t+1}, R_{t+1}) , then performs the same deterministic max on the realized S_{t+1} . The unit of update is the quadruple $(S_t, A_t, R_{t+1}, S_{t+1})$.

2.4 Running-Mean Update

The Q-learning target is one noisy sample of TQ . By the same statistical move used in the SARSA(0) report (which we do not redo here), the running-mean update steps $Q(S_t, A_t)$ toward the new sample with step size α :

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)).$$

The quantity in the bracket is the *Q-learning TD error*

$$\delta_t = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t).$$

When Q reaches the fixed point of T (i.e. $Q = q^*$), this error has expectation zero. While Q is still improving, the error has nonzero expectation in the direction that moves Q toward q^* .

3 Off-Policy: What the Change of Operator Bought Us

The Q-learning update references no policy. The target $R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$ depends on:

- the fixed input (S_t, A_t) at which we are updating,
- the sampled (S_{t+1}, R_{t+1}) from the environment,
- the current Q table.

None of these reference the policy that chose A_t . The agent still needs *some* policy to choose A_t , typically ε -greedy w.r.t. Q to ensure exploration, but that policy enters only as a data-generation mechanism. Different choices of behaviour policy produce different trajectories but the same target formula. As long as every (s, a) keeps getting visited, the samples are valid realizations of TQ .

This is the structural meaning of *off-policy*: the policy that generated the data (the behaviour policy) is separable from the policy whose value the target is estimating (here, the implicit greedy policy w.r.t. the current Q). The two can differ. Contrast with SARSA(0), where the target $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$ contains $A_{t+1} \sim \pi$ explicitly: changing π changes the target. Behaviour and target are tied. On-policy.

The off-policy property is not a clever trick. It falls out of the algebra: the optimality operator T contains no policy expectation, so when we sample it, no policy expectation survives in the target.

4 The Algorithm

Algorithm 1: Q-Learning

Input: step size $\alpha \in (0, 1]$, exploration $\varepsilon > 0$, discount γ , number of episodes N
Initialize $Q(s, a)$ arbitrarily for all s, a , with $Q(\text{terminal}, \cdot) = 0$
for each episode $e = 1, \dots, N$ **do**
 Initialize S_t at the episode start
 while S_t is not terminal **do**
 Choose $A_t \sim \mu(\cdot \mid S_t)$ from any behaviour policy μ with $\mu(a \mid s) > 0$ for all a (e.g. ε -greedy w.r.t. Q)
 Execute A_t ; observe R_{t+1}, S_{t+1}
 $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t))$
 $S_t \leftarrow S_{t+1}$
 end
end

Two structural observations versus SARSA(0).

No A_{t+1} in the loop. The body does not draw a next action. The target on line 7 uses the max over the action set at S_{t+1} , not a sample from any policy. The only action draw in the loop is A_t , drawn from the behaviour policy at the start of each iteration.

The behaviour policy is decoupled from the target. Line 5 just says “draw from any μ with full support.” Standard practice is ε -greedy w.r.t. the current Q (this is what ties Q-learning to the same coverage guarantee as SARSA), but any policy with $\mu(a \mid s) > 0$ for every (s, a) keeps the convergence story intact. SARSA’s algorithm box, by contrast, ties the action-selection step on line 6 to the policy that appears in the target.

5 Convergence

Q-learning converges to q^* under the standard two conditions: every state-action pair is visited infinitely often, and the step sizes satisfy the Robbins-Monro conditions

$$\sum_{t=0}^{\infty} \alpha_t = \infty, \quad \sum_{t=0}^{\infty} \alpha_t^2 < \infty.$$

Notably, the GLIE condition required for SARSA is *not* needed here. SARSA needed GLIE because its target tracks q_π for the current policy, so the policy itself had to converge to greedy for the limit q_π to equal q^* . Q-learning’s target directly references $\max_{a'} Q(S_{t+1}, a')$, which corresponds to the greedy policy w.r.t. the current Q already. Whatever behaviour policy is being followed, the operator being sampled is T , and T ’s fixed point is q^* .

Key Result

Q-learning convergence. Under coverage and Robbins-Monro step sizes, the Q-learning iterates satisfy

$$Q_t(s, a) \xrightarrow{t \rightarrow \infty} q^*(s, a) \quad \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$$

with probability one, for any behaviour policy μ that visits every pair infinitely often. Proof: Watkins (1989), Watkins and Dayan (1992); the argument is stochastic approximation on the contraction T on $\mathcal{Q}$.

The convergence statement is structural and follows the same pattern as SARSA(0): the running-mean update inherits the contraction of T in expectation, the Robbins-Monro conditions control the noise, and coverage ensures every entry of the table keeps being refreshed. We defer the formal stochastic-approximation machinery to the stochastic approximation report.